Matura-Übungssatz POS

Zwei komplette Übungs-Maturen im Stil der ehemaligen Beispiele. Jede Matura besteht aus 3 Beispielen zu je ca. 2 Stunden Bearbeitungszeit. In den 4 bis 4,5 Stunden Prüfungszeit ist nicht zu erwarten, dass alle drei Beispiele vollständig gelöst werden. Die Punktevergabe ist so gestaltet, dass eine sehr gute Note auch mit zwei vollständig und einem teilweise gelösten Beispiel erreichbar ist.

Allgemeine Hinweise für beide Varianten:

- Für Datenbanken wird ausschließlich linq2db mit SQLite verwendet.
- Für Netzwerk-Kommunikation wird die vorgegebene Datei `Transfer_2026.cs` verwendet. Es wird in einer einzigen Klasse pro Verbindung kommuniziert (mit einem `MsgType`-Enum als Diskriminator).
- Anstatt Custom Controls aus einer DLL zu laden, werden in dieser Übungsmatura eigene UserControls erstellt (mit DependencyProperties). Das deckt dasselbe Lernziel ab.
- Die Grammatik wird in der allgemeinen Backus-Naur-Form (ABNF) erstellt.

Matura Variante A

Beispiel A1 – Skigebiet-Simulator

Abschlussbedingungen

Beschreibung

In einem Skigebiet bewegen sich mehrere Skifahrer parallel zwischen den Liften und den Pisten. Jeder Lift hat eine begrenzte Kapazität, die durch eine Semaphore abgebildet wird. Skifahrer wählen ausgehend von einem Startpunkt den schnellsten Weg zu einem zufällig vergebenen Ziel im Gebiet. Die Wegfindung soll mit dem Dijkstra-Algorithmus über einen Graphen aus Knoten (Liftstationen) und Kanten (Lifte und Pisten mit jeweiliger Fahrzeit) erfolgen.

Aufgabe

Erstellen Sie ein WPF-Programm, das eine eigene UserControl namens `SkigebietAnzeige` enthält. Die UserControl rendert intern auf einem Canvas und stellt folgende DependencyProperties zur Verfügung: `Knoten` (eine Liste von Knoten-Objekten), `Kanten` (eine Liste von Kanten-Objekten) und `Skifahrer` (eine Liste von Skifahrer-Objekten). Bei Änderung der Properties soll die Anzeige automatisch aktualisiert werden.

Knoten werden als beschriftete Ellipsen gezeichnet, Kanten als Linien zwischen den Ellipsen (Lifte z.B. blau, Pisten rot oder schwarz). Skifahrer werden als farbige Ellipsen an ihrer aktuellen Position dargestellt. Wartende Skifahrer am Lift sollen als kleine Schlange neben der Liftstation sichtbar sein.

Laden Sie die Skigebiets-Daten aus einer XML-Datei (Stationen mit Koordinaten und Namen, Lifte mit Kapazität und Fahrzeit, Pisten mit Fahrzeit). Eine Beispiel-XML-Datei liegt in den Angabe-Dateien.

Ermöglichen Sie über einen Menüpunkt das Hinzufügen einer einstellbaren Anzahl an Skifahrern. Jeder Skifahrer soll als eigener Thread laufen und sich auf der `SkigebietAnzeige` entsprechend bewegen. Die Skifahrer dürfen sich nicht gegenseitig blockieren und die Lift-Kapazitäten dürfen nie überschritten werden. Verwenden Sie für die Lifte das Boarding-Pattern mit einer Semaphore. Solange ein Lift voll ist, sollen Skifahrer in einer Warteschlange am Lift sichtbar sein.

Implementieren Sie den Dijkstra-Algorithmus, der für jeden Skifahrer den schnellsten Weg vom aktuellen Knoten zu einem zufällig gewählten Ziel berechnet. Die Berechnung soll im Hintergrund (z.B. über den ThreadPool) laufen, damit die GUI flüssig bleibt. Die berechnete Route soll auf der Anzeige farblich passend zum Skifahrer eingezeichnet werden.

Sobald ein Skifahrer sein Ziel erreicht, soll er ein neues zufälliges Ziel bekommen. Ein Statusfeld zeigt für jeden Skifahrer den aktuellen Knoten, das Ziel und die Anzahl der bisherigen Fahrten. Bei Programm-Schließen sollen alle Threads sauber beendet werden (kein Deadlock).

Beispiel-Statistik nach 60 Sekunden:

```
Skifahrer 1: 4 Fahrten, aktuell Lift "Bergstation Süd"
Skifahrer 2: 6 Fahrten, aktuell Piste "Talabfahrt"
Skifahrer 3: 5 Fahrten, wartend an Lift "Sesselbahn 1" (Position 2)
```

Beispiel A2 – Bibliotheks-Verwaltung

Abschlussbedingungen

Beschreibung

Eine Bibliothek soll über ein Netzwerk-Programm verwaltet werden. Ein Server hält eine SQLite-Datenbank mit Büchern, Genres und Autoren. Mehrere Clients können gleichzeitig nach Büchern suchen, neue Bücher anlegen oder bestehende ausleihen. Die Kommunikation läuft über die vorgegebene Klasse `Transfer<T>`.

Aufgabe

Erstellen Sie ein WPF-Programm mit Server- und Client-Variante (zwei separate Projekte oder ein Projekt mit Auswahl beim Start). Verwenden Sie linq2db mit SQLite für die Datenbank. Das Datenmodell besteht aus den Tabellen `Buch` (Id, Titel, AutorId, GenreId, Verfuegbar), `Autor` (Id, Name) und `Genre` (Id, Name, ParentGenreId). Die Genre-Hierarchie ist also baumartig (z.B. "Roman" hat Kinder "Krimi" und "Fantasy").

Beim ersten Start des Servers soll die Datenbank aus den mitgelieferten CSV-Dateien (`autoren.csv`, `genres.csv`, `buecher.csv`) befüllt werden. Eine MessageBox soll bei erfolgreichem Import die Anzahl der importierten Bücher anzeigen.

Definieren Sie eine Klasse `BibliothekMessage`, die alle Nachrichtentypen abbildet und über `Transfer<BibliothekMessage>` versendet werden kann. Mögliche Nachrichtentypen sind über ein Enum `MsgType` definiert: `Suchen`, `Suchergebnis`, `Ausleihen`, `AusleiheBestaetigung`, `NeuesBuch`. Die Klasse enthält die für alle Typen relevanten Properties.

Im Client soll links ein TreeView die Genre-Hierarchie aus der Datenbank anzeigen (per LINQ-Abfrage rekursiv aufgebaut). Bei Klick auf ein Genre soll der Server eine Suche durchführen und alle Bücher dieses Genres und seiner Untergenres zurückliefern. Die Ergebnisse werden rechts in einer ListBox angezeigt. Zusätzlich gibt es ein Suchfeld für die Volltextsuche im Buchtitel.

Im Server soll mittig ein Log angezeigt werden, in dem alle eingehenden Nachrichten mit Zeitstempel und Client-IP angezeigt werden. Pro verbundenem Client soll ein eigener `Transfer<BibliothekMessage>` instanziiert werden.

Im Client soll ein gewähltes Buch über einen Button "Ausleihen" entlehnt werden können. Der Server prüft die Verfügbarkeit, aktualisiert die Datenbank und schickt eine Bestätigung oder Fehlermeldung zurück. Die Verfügbarkeit aktualisiert sich auch bei allen anderen verbundenen Clients automatisch.

Beispiel A3 – Schildkröten-Grafik

Abschlussbedingungen

Beschreibung

Es soll eine kleine Programmiersprache zur Steuerung einer "Schildkröte" auf einem Zeichenbrett umgesetzt werden. Die Schildkröte hinterlässt eine farbige Linie wenn der Stift unten ist und kann mit Befehlen wie Vorwärts, Drehen, Stift heben, Stift senken und Wiederholungen gesteuert werden. Damit lassen sich geometrische Muster zeichnen. Beispielprogramme zum Testen liegen in den Angabe-Dateien.

Aufgabe

Der Schildkröten-Interpreter soll folgende Befehle unterstützen:

- Vorwärts und Rückwärts gehen (mit Pixelangabe)
- Nach Links oder Rechts drehen (mit Gradangabe)
- Stift heben oder senken
- Stiftfarbe setzen (auf einen von 8 Namen: ROT, GRUEN, BLAU, GELB, SCHWARZ, WEISS, ORANGE, VIOLETT)
- Block (mehrere Befehle in `{ }` zusammenfassen)
- WIEDERHOLE n-mal einen Block

Erstellen Sie eine Grammatik in der allgemeinen Backus-Naur-Form (ABNF), die die Beispiel-Programme der Angabe abbilden kann.

Planen Sie die für die Umsetzung der Grammatik notwendigen Klassen als UML-Diagramm, wenn das Interpreter-Software-Design-Pattern als Basis verwendet werden soll. Die Basisklasse `Anweisung` mit der abstrakten Methode `Ausfuehren(SchildkroeteContext ctx)` und die Klasse `SchildkroeteContext` stehen in den Angabe-Dateien zur Verfügung.

Erstellen Sie ein C#-WPF-Programm mit einer eigenen UserControl namens `ZeichenBrett`, die intern auf einem Canvas basiert. Die UserControl stellt folgende öffentliche Methoden zur Verfügung: `LinieZeichnen(x1, y1, x2, y2, Brush farbe)`, `SchildkroeteSetzen(x, y, winkelGrad)` (zeichnet die Schildkröte als kleines Dreieck mit RotateTransform in Bewegungsrichtung), `Loeschen()` (entfernt alle Linien). Die Schildkröte selbst bleibt beim Löschen stehen.

Das Programm soll ein Schildkröten-Programm aus einer Textdatei laden können (über einen Filedialog). Parsen Sie die Eingabe und erstellen Sie eine Baumstruktur die der Eingabe entspricht. Bei Syntaxfehlern soll eine aussagekräftige Fehlermeldung mit Zeilennummer angezeigt werden. Verwenden Sie die Beispieldateien `Fehler1.txt` und `Fehler2.txt` zum Testen.

Werten Sie nach erfolgreichem Parsen die Baumstruktur aus und zeigen Sie die Bewegungen der Schildkröte am `ZeichenBrett` an. Zwischen den einzelnen Schritten sollen jeweils 100 Millisekunden Pause sein, damit der Zeichenvorgang nachvollziehbar ist. Die Animation darf die GUI nicht blockieren.

Beispielprogramm 1 (Quadrat):

```
STIFT SENKEN
WIEDERHOLE 4 {
    VORWAERTS 100
    RECHTS 90
}
```

Beispielprogramm 2 (Stern aus 5 Dreiecken):

```
FARBE ROT
STIFT SENKEN
WIEDERHOLE 5 {
    WIEDERHOLE 3 {
        VORWAERTS 80
        LINKS 120
    }
    LINKS 72
}
```

Matura Variante B

Beispiel B1 – Werkstatt-Simulator

Abschlussbedingungen

Beschreibung

In einer Kfz-Werkstatt nehmen mehrere Mechaniker (Threads) Aufträge aus einer gemeinsamen Auftragsschlange entgegen und bearbeiten diese. Die Aufträge unterscheiden sich in Bearbeitungsdauer und Priorität. Die Synchronisation zwischen Auftragseingang (Producer) und Mechanikern (Consumer) erfolgt über `Monitor.Wait` und `Monitor.PulseAll`. Eine WPF-Visualisierung zeigt die Auslastung der Werkstatt in Echtzeit.

Aufgabe

Erstellen Sie ein WPF-Programm mit drei Bereichen:

- Links die Auftragsschlange als ListBox
- Mittig die Mechaniker als animierte Rechtecke (mit Farb-Statusanzeige: grau wenn idle, gelb wenn arbeitend, grün wenn fertig)
- Rechts eine Statistik mit Gesamtaufträge, Erledigt, Durchschnittliche Wartezeit

Über einen Button "Auftrag generieren" sollen einzelne Aufträge erzeugt werden, über einen Button "Stress-Test" sollen 50 Aufträge in 5 Sekunden erzeugt werden. Jeder Auftrag hat eine Bearbeitungsdauer (zwischen 500 und 3000 ms) und eine Priorität (1 bis 3).

Implementieren Sie eine Klasse `AuftragsZentrale`, die intern eine Warteschlange hält und mit `Monitor.Wait` und `Monitor.PulseAll` arbeitet. Die Methode `NaechsterAuftrag()` blockiert solange kein Auftrag verfügbar ist. Aufträge mit höherer Priorität müssen vor Aufträgen mit niedrigerer Priorität ausgegeben werden, auch wenn die niedrigeren früher eingegangen sind.

Die Anzahl der Mechaniker (Threads) soll über einen Slider zwischen 1 und 8 einstellbar sein. Beim Erhöhen werden zusätzliche Threads gestartet, beim Verringern werden überschüssige Threads sauber beendet (über ein `CancellationToken` oder ein flag-basiertes System). Das Programm darf bei mehrfachem Auf- und Abwählen nicht abstürzen oder hängen.

Implementieren Sie einen einfachen Auftragsoptimierer: Wenn die durchschnittliche Wartezeit über 2 Sekunden steigt, soll automatisch ein zusätzlicher Mechaniker hinzugefügt werden (bis zum Maximum). Sinkt die durchschnittliche Wartezeit unter 0,5 Sekunden, wird einer entfernt (bis zum Minimum von 1).

Bei Programm-Schließen sollen alle Mechaniker-Threads sauber beendet werden, auch wenn sie gerade einen Auftrag bearbeiten oder im `Wait` blockiert sind.

Beispiel B2 – Stadtplan-Routenplaner

Abschlussbedingungen

Beschreibung

Ein Routenplaner soll für eine Stadt die kürzeste Route zwischen zwei Punkten finden. Die Stadt wird als gerichteter Graph mit Knoten (Kreuzungen) und Kanten (Straßen mit Länge in Metern und durchschnittlicher Geschwindigkeit) modelliert und in einer SQLite-Datenbank gespeichert. Die Wegfindung erfolgt mit dem Dijkstra-Algorithmus.

Aufgabe

Erstellen Sie ein WPF-Programm, das eine SQLite-Datenbank mit linq2db verwendet. Datenmodell: `Knoten` (Id, Name, Longitude, Latitude), `Kante` (Id, VonKnotenId, NachKnotenId, LaengeMeter, MaxKmh). Beim ersten Start wird die Datenbank aus den mitgelieferten CSV-Dateien (`knoten.csv`, `kanten.csv`) befüllt.

Im Hauptfenster soll links ein Canvas die Stadtkarte anzeigen. Knoten werden als Ellipsen gezeichnet, Kanten als Linien zwischen den Ellipsen. Verwenden Sie eine Methode `LonLatZuCanvas(lon, lat)` zur Umrechnung der geografischen Koordinaten in Pixelkoordinaten. Die Karte soll bei Größenänderung des Fensters mitskalieren.

Rechts soll ein Bedienfeld mit folgenden Elementen sein: ComboBox für Startknoten (per LINQ-Query befüllt), ComboBox für Zielknoten, Button "Route berechnen", Label für die berechnete Distanz und Fahrzeit, ListBox mit den Knoten der Route in Reihenfolge.

Implementieren Sie den Dijkstra-Algorithmus zur Berechnung der kürzesten Route nach Distanz. Die berechnete Route wird am Canvas durch eine farbige Verbindungslinie eingezeichnet. Die Fahrzeit ergibt sich aus `LaengeMeter / (MaxKmh / 3.6)` pro Kante.

Über eine ComboBox kann zwischen den Optimierungszielen "Kürzeste Distanz" und "Schnellste Fahrzeit" gewählt werden. Beide Optionen müssen über denselben Dijkstra-Code laufen, der die Gewichtung der Kanten parametrisiert bekommt.

Klick auf einen Knoten am Canvas soll diesen als Startknoten setzen, Rechtsklick als Zielknoten. Knoten ohne Verbindung zum Startknoten (nicht erreichbar) sollen nach der Berechnung optisch hervorgehoben werden (z.B. graue Ellipse).

Über einen Menüpunkt "Statistik" soll per LINQ-Query angezeigt werden: Anzahl Knoten, Anzahl Kanten, durchschnittliche Kantenlänge, längste und kürzeste Kante mit den jeweiligen Knotennamen.

Beispiel B3 – Smart-Home-Skript

Abschlussbedingungen

Beschreibung

Ein Smart-Home-System soll über kleine Skripte gesteuert werden. Die Skripte werden auf einem Client geschrieben und an den Server (das Smart-Home-Gateway) gesendet, der sie ausführt und die Effekte an alle verbundenen Clients zurückspiegelt. Der Server simuliert mehrere Räume mit Lampen und Sensoren.

Aufgabe

Die Smart-Home-Sprache soll folgende Befehle unterstützen:

- Lampe einschalten oder ausschalten (mit Raum- und Lampenname)
- Lampe dimmen (mit Wert von 0 bis 100)
- Warten (mit Sekundenzahl)
- WENN Sensor (Temperatur, Helligkeit) Vergleich (KLEINER, GLEICH, GROESSER) Wert DANN Block
- Block (mehrere Befehle in `{ }` zusammenfassen)

Erstellen Sie eine Grammatik in der allgemeinen Backus-Naur-Form (ABNF), die alle Beispiel-Skripte der Angabe abbilden kann.

Planen Sie die für die Umsetzung der Grammatik notwendigen Klassen als UML-Diagramm, wenn das Interpreter-Software-Design-Pattern als Basis verwendet werden soll. Die Basisklasse `Anweisung` mit der Methode `Ausfuehren(SmartHomeContext ctx)` und die Klasse `SmartHomeContext` stehen in den Angabe-Dateien zur Verfügung.

Definieren Sie eine Klasse `SmartHomeMessage` mit einem Enum `MsgType` (`SkriptSenden`, `LampenStatus`, `SensorWert`, `Fehler`). Diese Klasse wird über die vorgegebene Klasse `Transfer<SmartHomeMessage>` zwischen Server und Client ausgetauscht.

Erstellen Sie ein WPF-Programm mit Server- und Client-Modus (Auswahl beim Programmstart). Im Server werden die Räume mit Lampen und Sensoren aus der Datei `raeume.xml` geladen. Eine grafische Übersicht zeigt für jeden Raum die aktuellen Lampenzustände und Sensorwerte. Sensorwerte werden im Server alle 2 Sekunden zufällig leicht verändert (simuliert Messungen) und an alle Clients verteilt.

Im Client soll eine TextBox zur Skript-Eingabe vorhanden sein, sowie ein Button "Senden". Bei Klick wird das Skript geparst (Syntaxprüfung am Client), und falls korrekt, mit `Transfer.Send` an den Server geschickt. Bei Syntaxfehlern wird die Fehlerstelle im Skript farblich markiert und eine Meldung angezeigt. Vom Server eingehende Statusupdates werden in einer Übersicht (TreeView) live angezeigt.

Im Server soll das empfangene Skript geparst und ausgeführt werden. Bei `WARTE`-Befehlen darf der Server nicht blockiert sein, andere Clients müssen weiter Skripte senden können. Verwenden Sie dafür einen eigenen Thread oder ThreadPool pro Skript. Nach jedem Befehl werden die geänderten Zustände an alle Clients verteilt.

Beispielskript:

```
LAMPE Wohnzimmer Decke EIN
DIMMEN Wohnzimmer Decke 60
WARTE 3
WENN Helligkeit Wohnzimmer KLEINER 20 DANN {
    LAMPE Wohnzimmer Decke EIN
    DIMMEN Wohnzimmer Decke 100
}
```

Punktesystem

Insgesamt sind pro Matura **100 Punkte** zu erreichen. Die Verteilung pro Beispiel:

Matura Variante A

Beispiel A1 – Skigebiet-Simulator (35 Punkte)

Teilaufgabe	Punkte
Eigene UserControl `SkigebietAnzeige` mit DependencyProperties	4
XML-Datei korrekt geladen, Knoten und Kanten in Datenstruktur abgebildet	4
Skifahrer als Threads erzeugbar, Anzahl einstellbar	4
Lift-Kapazität korrekt mit Semaphore umgesetzt (Boarding-Pattern)	5
Warteschlange wird in der UserControl sichtbar	3
Dijkstra-Algorithmus korrekt implementiert	7
Routen werden farblich passend visualisiert	3
Threads werden bei Programm-Schließen sauber beendet, kein Deadlock	3
Statusfeld mit aktueller Information pro Skifahrer	2

Beispiel A2 – Bibliotheks-Verwaltung (35 Punkte)

Teilaufgabe	Punkte
linq2db mit SQLite, Datenmodell mit drei Tabellen korrekt	4
CSV-Import mit korrekter Anzeige der importierten Anzahl	3
`BibliothekMessage`-Klasse mit `MsgType`-Enum, alle Felder	4
Server-Client-Verbindung über `Transfer<T>` funktioniert mit mehreren Clients	5
TreeView baut Genre-Hierarchie korrekt aus DB auf (rekursiv per LINQ)	5
Suche nach Genre liefert auch Bücher der Untergenres	4
Volltextsuche im Buchtitel funktioniert	2
Server-Log mit Zeitstempel und Client-IP	2
Ausleihen aktualisiert DB und benachrichtigt alle Clients	4
Saubere Fehlerbehandlung bei nicht verfügbaren Büchern	2

Beispiel A3 – Schildkröten-Grafik (30 Punkte)

Teilaufgabe	Punkte
ABNF-Grammatik vollständig und konsistent	5
UML-Diagramm der Interpreter-Klassen passend	4
Eigene UserControl `ZeichenBrett` mit Methoden + Schildkröte als Dreieck mit RotateTransform	5
Textdatei laden über Filedialog	1
Parser baut Baumstruktur korrekt auf	5
Aussagekräftige Syntaxfehler mit Zeilennummer	3
Ausführung der Baumstruktur mit Zeichnung am Brett	4
100 ms Pause zwischen Schritten, GUI bleibt responsiv	3

Matura Variante B

Beispiel B1 – Werkstatt-Simulator (35 Punkte)

Teilaufgabe	Punkte
WPF-Layout mit drei Bereichen (Schlange, Mechaniker, Statistik)	3
Aufträge generieren (Einzeln + Stress-Test)	3
`AuftragsZentrale` mit `Monitor.Wait` und `Monitor.PulseAll`	6
Prioritäts-Warteschlange (höhere Priorität zuerst)	4
Mechaniker-Threads über Slider dynamisch hinzufügen und entfernen	5
Mechaniker-Visualisierung mit Farbänderung (grau, gelb, grün)	3
Statistik korrekt berechnet (Durchschn. Wartezeit, etc.)	3
Auftrags-Optimierer fügt Mechaniker dynamisch hinzu/entfernt sie	4
Saubere Beendigung aller Threads beim Programm-Schließen	4

Beispiel B2 – Stadtplan-Routenplaner (35 Punkte)

Teilaufgabe	Punkte
linq2db mit SQLite, Datenmodell `Knoten` und `Kante`	3
CSV-Import beim ersten Start	2
Karte am Canvas zeichnen (Knoten als Ellipsen, Kanten als Linien)	4
Koordinatenumrechnung Lon/Lat zu Canvas-Pixel	3
Bedienfeld mit ComboBoxen (Start, Ziel) per LINQ befüllt	3
Dijkstra-Algorithmus korrekt implementiert	7
Route am Canvas farblich eingezeichnet	2
Distanz und Fahrzeit korrekt angezeigt	2
Optimierungsziel (Distanz vs. Fahrzeit) umschaltbar	3
Klick und Rechtsklick auf Knoten setzt Start/Ziel	2
Nicht erreichbare Knoten werden hervorgehoben	2
Statistik-Menüpunkt mit korrekten LINQ-Abfragen	2

Beispiel B3 – Smart-Home-Skript (30 Punkte)

Teilaufgabe	Punkte
ABNF-Grammatik vollständig und konsistent	4
UML-Diagramm der Interpreter-Klassen passend	3
`SmartHomeMessage`-Klasse mit `MsgType`-Enum	2
Server-Client-Verbindung über `Transfer<T>`	3
Server lädt Räume aus XML, Sensorwerte werden simuliert	3
Parser baut Baumstruktur korrekt auf	5
Syntaxfehler werden im Client farblich markiert	2
Ausführung am Server, Statusupdates an alle Clients	4
`WARTE` blockiert Server nicht, paralleles Ausführen mehrerer Skripte	2
TreeView am Client zeigt Live-Status	2

Bewertungsschlüssel

Aufgrund der knappen Zeit (4 bis 4,5 Stunden für 3 Beispiele zu je ca. 2 Stunden) ist eine vollständige Lösung aller Beispiele nicht zu erwarten. Die folgende Skala berücksichtigt das:

Note	Punkte (von 100)
Sehr gut (1)	86 bis 100
Gut (2)	71 bis 85
Befriedigend (3)	56 bis 70
Genügend (4)	41 bis 55
Nicht genügend (5)	0 bis 40

Innerhalb der Teilaufgaben werden Teilpunkte vergeben (z.B. 4 von 6 Punkten, wenn die Synchronisation grundsätzlich funktioniert, aber unter Last selten zu kurzen Hängern kommt). Schwerwiegende Fehler (Programm-Absturz, Datenverlust, falsche Ergebnisse) führen zu 0 Punkten der jeweiligen Teilaufgabe, auch wenn Code geschrieben wurde.

Zusatzpunkte werden nicht vergeben. Funktionalität, die in der Angabe nicht gefordert ist, wird nicht bewertet, auch wenn sie sinnvoll oder beeindruckend ist (Strategie: erst alle Pflichtteile, dann Polish).

Empfohlene Reihenfolge zur Bearbeitung

Beide Maturen sind so aufgebaut, dass das schwächste der drei Beispiele am Ende stehen kann. Empfohlene Reihenfolge:

1. Mit dem Beispiel beginnen, in dem du dich am sichersten fühlst (oft Datenbank/LINQ oder Multithreading)
2. Im zweiten Slot den Interpreter, da hier ABNF und UML als Teilpunkte schon ohne lauffähigen Parser zählen
3. Das verbleibende Beispiel zum Schluss, mit dem Risiko es nicht ganz fertig zu schaffen

Nach 1,5 Stunden pro Beispiel sollten die Pflichtteile stehen, die letzten 30 Minuten gehen für Polish und Fehlerbehebung drauf. Bei Zeitnot lieber ein Beispiel komplett liegen lassen und in den anderen die Punkte holen, statt überall halbfertig zu enden.